

Lab 8: Vision-Guided Pick and Place

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

0.1	Objectives	2
1	Part 1: Camera Calibration	3
1.1	Why Calibrate?	3
1.2	Calibration Grid Setup	3
1.3	Camera Data Setup	3
1.4	Grid Calibration Procedure	3
2	Part 2: Creating a Vision Process	5
2.1	What Is a Vision Process?	5
2.2	Set Up a 2D Locator	5
2.3	Train the Vision Model	5
3	Part 3: Vision Commands in TP Programs	6
3.1	Key Commands	6
3.2	How It Works	6
4	Part 4: Vision-Guided Pick and Place Program	7
4.1	Setup	7
4.2	The Program	7
4.3	Accuracy Test	8
4.4	Observations	8
5	Quick Reference	9
5.1	Vision Commands	9
5.2	Calibration Summary	9
5.3	Lab Completion	9

Safety Reminder

- Start in **T1 mode** — max 250 mm/s
- Hold the **Deadman Switch** at all times
- Always test new programs in **single-step** mode first
- The camera is fixed above the workspace — do not bump or reposition it

Prerequisites

This lab builds on Labs 5–7. You should be comfortable with jogging, teaching positions, UTOOL/UFRAME setup, registers, offsets, and FOR loops.

0.1 Objectives

In this lab you will:

1. Understand how the fixed camera relates to the robot's coordinate system
2. Calibrate the iRVision fixed camera using the calibration grid
3. Create a vision process to detect a part on the workspace
4. Write a TP program that uses vision to locate and pick parts at unknown positions

DRAFT NOTE — TO REFINE: The exact menu paths, button sequences, and screen names in Parts 1–3 need to be verified against the actual controller after performing the calibration. Placeholder steps marked with **[VERIFY]** should be updated with the real pendant navigation.

1 Part 1: Camera Calibration

1.1 Why Calibrate?

Camera-to-Robot Coordinate Transform

The camera sees pixels. The robot works in millimeters. Calibration computes the transformation between camera image coordinates and robot world coordinates, so that when the camera says “the part is at pixel (320, 240),” the robot knows the corresponding (X, Y) in its workspace. For a **fixed camera** (mounted above the workspace, not on the robot), this is a one-time calibration — as long as the camera doesn’t move.

1.2 Calibration Grid Setup

Exercise: Prepare for Calibration

1. Locate the FANUC calibration grid (black dots on white background)
2. Place the grid **flat** on the workspace, centered under the camera
3. Ensure the grid is well-lit with no shadows or reflections
4. Verify the camera can see the entire grid (check on the pendant vision screen)

1.3 Camera Data Setup

[VERIFY] The exact menu path may differ on our controller version.

Teach Pendant

MENU → **iRVision** → **Camera Data** → **Create**

Set camera type: **Fixed Camera**

Name it (e.g., FIXED_CAM1)

1.4 Grid Calibration Procedure

[VERIFY] Confirm the number of touch-up points and their order on our specific grid.

Exercise: Calibrate the Camera

1. In the camera data screen, select **Calibrate** → **Grid Calibration**
2. Snap an image of the grid — the system should detect the dot pattern
3. **Grid Frame Setup:** Touch the robot TCP to specific dots on the grid:
 - Orient Origin (reference dot)
 - X-direction point
 - Y-direction point
 - System Origin (center dot)

At each point: jog the TCP to touch the dot, then **SHIFT + F5 [RECORD]**
4. Press **Calibrate** — the system computes the camera-to-world transform
5. Check the calibration error — should be below 1 mm for good results

Record your calibration results:

Metric	Value
Calibration error (pixels)	

Metric	Value
Calibration error (mm)	
Camera height from grid (approx.)	

Exercise: Verify Calibration

1. Place a small object (e.g., a pen tip) on the workspace
2. Jog the robot TCP to touch the object, note the displayed position
3. On the vision screen, snap an image and check where the system thinks the object is
4. The vision-reported position should match the robot position within 1–2 mm

2 Part 2: Creating a Vision Process

2.1 What Is a Vision Process?

Vision Process

A vision process is a stored recipe that tells the camera **what to look for** and **how to find it**. It includes:

- Which camera to use (our calibrated fixed camera)
- A **locator tool** — the detection algorithm (we use GPM: Geometric Pattern Match)
- A **reference image** — a trained template of the part to find
- Search parameters — region of interest, score threshold, etc.

2.2 Set Up a 2D Locator

[VERIFY] Confirm the exact navigation and options for creating a vision process on our controller.

Teach Pendant

MENU → **iRVision** → **Vision Process** → **Create**

Name: FIND_PART

Camera: select your calibrated camera data

Locator tool: **GPM Locator** (Geometric Pattern Match)

2.3 Train the Vision Model

Exercise: Train the Part Model

1. Place the part in the workspace in a clear, well-lit position
2. In the vision process editor, snap a **reference image**
3. Draw a **region of interest** (ROI) around the part
4. The system extracts the geometric features as the model
5. Set the **match score threshold** (e.g., 70%) — parts with lower scores are rejected
6. Test: move the part to a different position, snap a new image, run **Find** — it should detect the part and show the match score

Record your results:

Test	Part position	Detected?	Score (%)	X (mm)	Y (mm)
1	Center				
2	Left side				
3	Right side				
4	Rotated 45°				

Troubleshooting

- **Low score:** Improve lighting, retrain model, or increase ROI
- **Not detected:** Part may be outside search region — enlarge the ROI
- **False detections:** Increase score threshold or reduce background clutter

3 Part 3: Vision Commands in TP Programs

3.1 Key Commands

[VERIFY] Confirm the exact instruction syntax and pendant menu paths for adding vision commands.

Command	Function
VISION RUN_FIND 'FIND_PART'	Run the vision process, snap image, find the part
VISION GET_OFFSET 'FIND_PART' VR[1]	Get the detected position offset into Vision Register
VISION GET_NFOUND 'FIND_PART' R[5]	Get the number of parts found into register R[5]

Teach Pendant

To add a vision command in a TP program:

[INST] → look for **Vision** or **VISION** submenu

Select the command type, then enter the vision process name.

3.2 How It Works

Vision Offset Flow

1. You teach a **reference pick position** P[1] with the part at a known location
2. The vision process is trained with the part at this same reference location
3. At runtime, the camera finds the part at its **actual** location
4. VISION GET_OFFSET computes the difference (offset) from reference to actual
5. You apply this offset to your taught position: L P[1] ... Offset,VR[1]
6. The robot picks from the correct location regardless of where the part was placed

4 Part 4: Vision-Guided Pick and Place Program

4.1 Setup

Exercise: Teach Reference Positions

1. Place the part at the **reference location** (where the vision model was trained)
2. Teach: **P[1]** = Pick approach (50mm above part)
3. Teach: **P[2]** = Pick (gripper at part)
4. Teach: **P[3]** = Place approach
5. Teach: **P[4]** = Place position
6. Teach: **P[5]** = Home

4.2 The Program

Exercise: Create Vision Pick-and-Place

Create a program called VISION_PICK. Test in single-step mode.

```
1: UT00L_NUM=2 ;
2: UFRAME_NUM=2 ;
3: DO[1:Gripper]=OFF ;
4: J P[5:Home] 100% FINE ;
5: LBL[1] ;
6:   ! Run vision to find part ;
7:   VISION RUN_FIND 'FIND_PART' ;
8:   VISION GET_NFOUND 'FIND_PART' R[5] ;
9:   IF R[5:Found]=0, JMP LBL[90] ;
10:  ! Part found – get offset ;
11:  VISION GET_OFFSET 'FIND_PART' VR[1] ;
12:  ! Pick with vision offset ;
13:  L P[1:Pick_App] 500mm/sec FINE Offset,VR[1] ;
14:  L P[2:Pick] 100mm/sec FINE Offset,VR[1] ;
15:  DO[1:Gripper]=ON ;
16:  WAIT .30(sec) ;
17:  L P[1:Pick_App] 500mm/sec FINE Offset,VR[1] ;
18:  ! Place (fixed location, no offset) ;
19:  L P[3:Place_App] 500mm/sec CNT50 ;
20:  L P[4:Place] 100mm/sec FINE ;
21:  DO[1:Gripper]=OFF ;
22:  WAIT .30(sec) ;
23:  L P[3:Place_App] 500mm/sec FINE ;
24:  J P[5:Home] 100% FINE ;
25:  JMP LBL[1] ;
26: LBL[90] ;
27: ! No part found ;
28: J P[5:Home] 100% FINE ;
```

Test Procedure

1. First run with the part at the **reference location** — the offset should be near zero
2. Move the part slightly (2–3 cm) and run again — the robot should follow
3. Gradually increase the displacement, testing at several positions across the workspace
4. Always run in **single-step** and **low speed** until you trust the offsets

4.3 Accuracy Test

Exercise: Measure Vision Accuracy

Place the part at 5 different locations in the workspace. For each, record where the vision says the part is and where the robot actually picks.

Test	Part location (approx.)	Vision offset X	Vision offset Y	Pick successful?
1	Center (reference)			
2	5 cm left			
3	5 cm forward			
4	Far corner			
5	Rotated 30°			

4.4 Observations

1. How does accuracy change as the part moves farther from the reference position?
2. What happens when the part is rotated? Does the robot compensate for rotation?
3. What is the approximate repeatability of the vision system (place the part in the same spot, run 3 times)?

5 Quick Reference

5.1 Vision Commands

Command	Pendant path	Function
VISION RUN_FIND	[INST] □ Vision	Run vision process
VISION GET_OFFSET	[INST] □ Vision	Get offset into VR
VISION GET_NFOUND	[INST] □ Vision	Get number of found parts
Offset, VR[1]	F4 on motion line	Apply vision offset to motion

5.2 Calibration Summary

Step	What to do
Grid placement	Flat on workspace, centered under camera, well-lit
Touch-up points	TCP to specific dots on grid (4 points)
Verify	Place object, compare vision position vs robot position

5.3 Lab Completion

- ☐ Calibrated the fixed camera using the grid (Part 1)
- ☐ Created and trained a vision process (Part 2)
- ☐ Tested vision commands in a TP program (Part 3)
- ☐ Completed vision-guided pick and place (Part 4)
- ☐ Measured vision accuracy across the workspace (Part 4)

Instructor Signature: _____

Date: _____